

UTS

PENGOLAHAN CITRA



NAMA : Hafizhul Faqih

NIM : 202431150

KELAS : A

DOSEN : Rizqia Cahyaningtyas, S.Kom, M.T

NO.PC : 16

ASISTEN : 1. Sharira Maharani

2. Muhammad Hanif Nuruddin Jabbar

3. Fhazel Kesra Arivi

4. Muhammad Farhan Fahrezy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2025/2026

DAFTAR ISI

Isi

DAFTAR ISI	2
BAB I	3
PENDAHULUAN	3
1.1 Rumusan Masalah	3
1.2 Tujuan Masalah	3
1.3 Manfaat Masalah	3
BAB II	4
LANDASAN TEORI	4
2.1. Citra digital	4
2.2. Pengolahan Citra Digital	4
2.3. Model Warna RGB dan Pemisahan Kanal Warna	4
2.4. Histogram Citra	4
2.5. Konversi Citra Grayscale	5
2.6. Segmentasi Citra Berbasis Thresholding	5
2.7. Peningkatan Kecerahan dan Kontras (Image Enhancement)	5
BAB III	6
HASIL	6
3.1. Pembahasan Library	6
3.2. Pembahasan Soal Pertama	6
3.3. Pembahasan Soal Kedua	11
3.4. Pembahasan Soal Ketiga	16
BAB IV	22
PENUTUP	22
DAFTAR PUSTAKA	23

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

- Bagaimana cara membaca dan menampilkan citra digital menggunakan *library* OpenCV dan Matplotlib dalam Python?
- Bagaimana cara memisahkan komponen warna (Merah, Hijau, dan Biru/RGB) dari sebuah citra dan menampilkan histogram masing-masing warna?
- Bagaimana cara melakukan konversi citra berwarna (RGB) menjadi citra *grayscale* secara manual menggunakan rumus pembobotan?
- Bagaimana cara menerapkan segmentasi citra berbasis nilai ambang (*thresholding*) pada masing-masing kanal warna untuk memisahkan objek tertentu?
- Bagaimana cara melakukan peningkatan kecerahan (*brightness*) dan kontras (*contrast*) pada citra *grayscale* dengan transformasi linear?

1.2 Tujuan Masalah

- Mampu membaca, memproses, dan menampilkan citra digital menggunakan *library* OpenCV (cv2) dan Matplotlib dalam lingkungan Python/Jupyter Notebook.
- Mampu memisahkan dan memvisualisasikan masing-masing warna (Biru, Hijau, Merah) dari citra RGB beserta histogram distribusi nilai *pixel* nya.
- Mampu mengonversi citra berwarna menjadi citra *grayscale* secara manual menggunakan rumus pembobotan *luminance*.
- Mampu melakukan segmentasi citra dengan teknik *thresholding* berbasis nilai ambang pada komponen warna tertentu untuk mendeteksi objek berwarna spesifik.
- Mampu meningkatkan kualitas citra *backlight* melalui operasi penambahan nilai *brightness* dan pengali kontras menggunakan transformasi linear sederhana.

1.3 Manfaat Masalah

- Memperoleh pemahaman praktis mengenai representasi dan struktur data citra digital dalam format matriks *pixel* tiga dimensi ($\text{tinggi} \times \text{lebar} \times \text{kanal warna}$).
- Dapat mengaplikasikan konsep ruang warna (*color space*) RGB dalam praktik pengolahan citra menggunakan Python.
- Dapat memahami hubungan antara distribusi intensitas *pixel* dan histogram citra, sehingga dapat menganalisis karakteristik visual suatu gambar secara kuantitatif.
- Mampu menerapkan teknik segmentasi berbasis *thresholding* sebagai dasar untuk pendeteksian objek berwarna dalam citra.
- Mampu menguasai teknik dasar perbaikan citra (image enhancement) melalui operasi *brightness* dan *contrast* yang menjadi pondasi dalam banyak aplikasi pengolahan citra nyata seperti penginderaan jauh, medis, dan industri.

BAB II

LANDASAN TEORI

2.1. Citra digital

Citra Digital merupakan representasi visual dari suatu objek nyata dalam bentuk matriks dua dimensi yang tersusun atas elemen terkecil yang disebut *pixel* (picture element). Setiap *pixel* menyimpan nilai intensitas yang merepresentasikan informasi warna atau kecerahan pada posisi tertentu dalam citra.

2.2. Pengolahan Citra Digital

Pengolahan citra digital atau PCD merupakan bidang yang melibatkan manipulasi dan analisis gambar menggunakan algoritma komputer. Pengolahan citra digital mengubah gambar menjadi format yang dapat dipahami dan dikerjakan komputer, memungkinkan berbagai peningkatan dan analisis yang tidak mungkin dilakukan dengan fotografi tradisional saja.

Dalam proses pengolahan citra digital terdapat penggunaan algoritma komputer untuk melakukan pemrosesan gambar pada gambar digital. Dimulai dari pemilihan objek yang ingin dilakukan proses citra digital, lalu objek akan ditangkap menggunakan berbagai sumber contohnya kamera, pemindai, atau sensor. Gambar yang telah diambil akan diubah menjadi format digital yang direpresentasikan sebagai matriks *pixel*, yang setiap *pixel*-nya berisi tentang informasi nilai warna dan kecerahan.

2.3. Model Warna RGB dan Pemisahan Kanal Warna

Model warna RGB (Red, Green, Blue) merupakan model yang paling banyak digunakan dalam pengolahan citra digital. Dalam model ini, setiap warna dibentuk oleh kombinasi tiga komponen primer: merah (R), hijau (G), dan biru (B). Pada implementasi OpenCV, urutan kanal disimpan dalam format BGR (Blue, Green, Red), sehingga untuk mendapatkan kanal biru diakses pada indeks 0, hijau pada indeks 1, dan merah pada indeks 2. Pemisahan kanal warna memungkinkan analisis distribusi intensitas masing-masing komponen secara terpisah, yang berguna dalam deteksi warna dan segmentasi objek spesifik dalam citra.

2.4. Histogram Citra

Histogram citra adalah representasi grafis yang menggambarkan distribusi frekuensi nilai intensitas *pixel* dalam suatu citra. Sumbu horizontal histogram mewakili nilai intensitas *pixel* (0–255 untuk citra 8-bit), sedangkan sumbu vertikal mewakili frekuensi kemunculan setiap nilai intensitas tersebut. Histogram memberikan informasi kuantitatif mengenai kecerahan, kontras, dan distribusi warna dalam. Dalam praktik, analisis histogram digunakan untuk menentukan kebutuhan perbaikan citra, seperti ekualisasi histogram atau penyesuaian kontras, serta sebagai dasar metode segmentasi berbasis *thresholding*.

2.5.Konversi Citra Grayscale

Konversi citra berwarna menjadi citra grayscale merupakan salah satu tahap pra-pemrosesan yang paling umum dilakukan dalam pengolahan citra digital. Proses ini mereduksi citra RGB tiga kanal menjadi satu kanal dengan nilai intensitas tunggal yang merepresentasikan kecerahan (luminance) setiap *pixel*. Rumus konversi yang digunakan dalam praktikum ini mengacu pada standar pembobotan perseptual yang memperhitungkan sensitivitas mata manusia terhadap setiap komponen warna, yaitu: $Y = 0.2989 \times R + 0.5870 \times G + 0.1141 \times B$.

2.6.Segmentasi Citra Berbasis Thresholding

Thresholding adalah salah satu metode segmentasi citra yang paling sederhana dan efisien, yang bekerja dengan mengklasifikasikan setiap *pixel* menjadi dua kelompok berdasarkan perbandingan nilai intensitasnya terhadap nilai ambang (*threshold*) tertentu. Pada segmentasi berbasis kanal warna (*color thresholding*), nilai ambang diterapkan secara terpisah pada masing-masing kanal warna (R, G, B) untuk mengisolasi objek dengan rentang warna tertentu dalam citra.

Pada praktikum ini, segmentasi dilakukan dengan menetapkan batas bawah dan batas atas nilai ambang untuk kanal biru (ambang_bawah_biru = 95, ambang_atas_biru = 255), merah (ambang_bawah_merah = 120, ambang_atas_merah = 255), dan hijau (ambang_bawah_hijau = 95, ambang_atas_hijau = 255)

2.7.Peningkatan Kecerahan dan Kontras (Image Enhancement)

Peningkatan kecerahan (brightness enhancement) dan kontras (contrast enhancement) merupakan teknik image enhancement yang bertujuan untuk memperbaiki kualitas visual citra sehingga lebih mudah diinterpretasikan.

Teknik ini sangat berguna dalam perbaikan citra backlight di mana objek tampak gelap karena pencahayaan latar yang berlebihan.

BAB III

HASIL

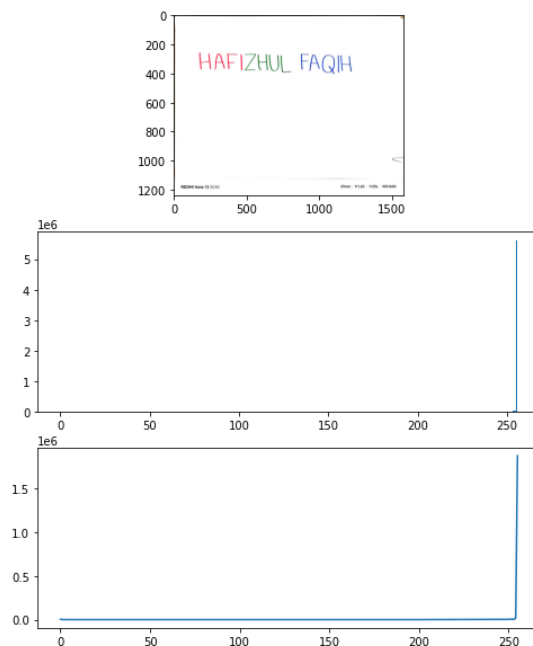
3.1. Pembahasan Library

- Numpy
gambar dianggap sebagai matriks (array) angka. NumPy digunakan untuk menyimpan data *pixel* gambar, melakukan manipulasi matriks (seperti memotong gambar/ROI), dan perhitungan aljabar linier.
- Opencv2
Menyediakan ratusan fungsi untuk pengolahan citra (image processing) dan visi komputer, seperti membaca gambar (imread), menampilkan gambar, deteksi tepi, deteksi wajah, dan memproses video.
- Matplotlib
Digunakan untuk menampilkan gambar, terutama di dalam Jupyter Notebook, atau saat perlu memplot histogram warna, grafik deteksi, dan plot hasil analisis lainnya.

3.2. Pembahasan Soal Pertama

Pembuatan Citra Kontras

```
In [3]: kontras = cv.cvtColor(gambar, cv.COLOR_BGR2RGB)
fig, axs = plt.subplots(3, figsize=(8,10))
axs[0].imshow(kontras)
axs[1].hist(kontras.ravel(),256,[0,256])
hist = cv.calcHist([kontras],[0],None,[256],[0,256])
axs[2].plot(hist)
plt.show()
```



Seperti ketentuan soal, langkah pertama ada pembuatan gambar dengan meningkatkan kontras. Dalam pengolahan citra digital, gambar dibaca seperti array 3D, yang dimensinya meliputi *height* (tinggi), *width* (lebar), dan warnanya

Kemudian kita akan menampilkan gambar yang telah ditingkatkan kontrasnya, histogram sebelum peningkatan kontras, dan histogram setelah kontrasnya ditingkatkan.

Terdapat perbedaan pada kedua histogram tersebut. Pada histogram pertama, garis biru yang merepresentasikan warna tampak tipis dalam diagram balok. Pada histogram kedua, garis biru tersebut mulai tampak bahkan dari sumbu 0, dan lebih tebal daripada kurva pertama dalam diagram garis.

Pembuatan Citra Biru

```
In [4]: b = gambar[:, :, 0]

fig, axs = plt.subplots(2,2, figsize=(10, 8))

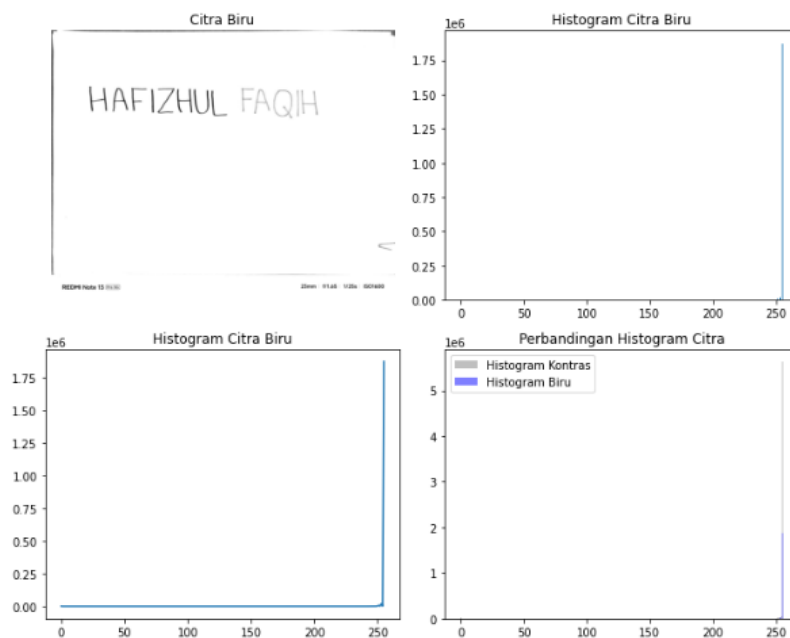
axs[0,0].imshow(b, cmap='gray')
axs[0,0].set_title("Citra Biru")
axs[0,0].axis('off')

axs[0,1].set_title("Histogram Citra Biru")
axs[0,1].hist(b.ravel(),256,[0,256])

axs[1,0].set_title("Histogram Citra Biru")
hist_blue = cv.calcHist([b],[0],[None],[256],[0,256])
axs[1,0].plot(hist_blue)

axs[1,1].hist(kontras.ravel(), bins=256, range=[0,256], color='gray', alpha=0.5, label="Histogram Kontras")
axs[1,1].hist(b.ravel(), bins=256, range=[0,256], color='blue', alpha=0.5, label="Histogram Biru")
axs[1,1].set_title("Perbandingan Histogram Citra")
axs[1,1].legend()

plt.tight_layout()
plt.show()
```



Pertama adalah pemilihan kanal warna, karena sebuah gambar dibaca dalam *array* karena yang membedakan warna pada sebuah citra adalah dimensi ketiga, maka pada *variable* *b* akan merepresentasikan satu warna yaitu warna biru (0).

Kemudian warna biru yang ada pada dimensi *array* *b* akan dibuat menjadi *grayscale* dengan `cmap='gray'`

Kemudian pada tiap histogram yang ditampilkan sebenarnya membawa informasi yang sama, yaitu distribusi nilai *pixel* pada kanal warna biru. Namun dengan visualiasi yang berbeda.

Histogram pertama merupakan representasi distribusi nilai kanal warna biru dalam bentuk diagram balok, histogram kedua merupakan representasi distribusi nilai kanal biru dalam diagram garis, dan histogram ketiga adalah perbandingan histogram biru yang dibuat *grayscale* dengan histogram yang merepresentasikan nilai warna dengan kontras (asli)

Pembuatan Citra Merah

```
In [5]: r = gambar[:, :, 2]

fig, axs = plt.subplots(2,2, figsize=(10, 8))

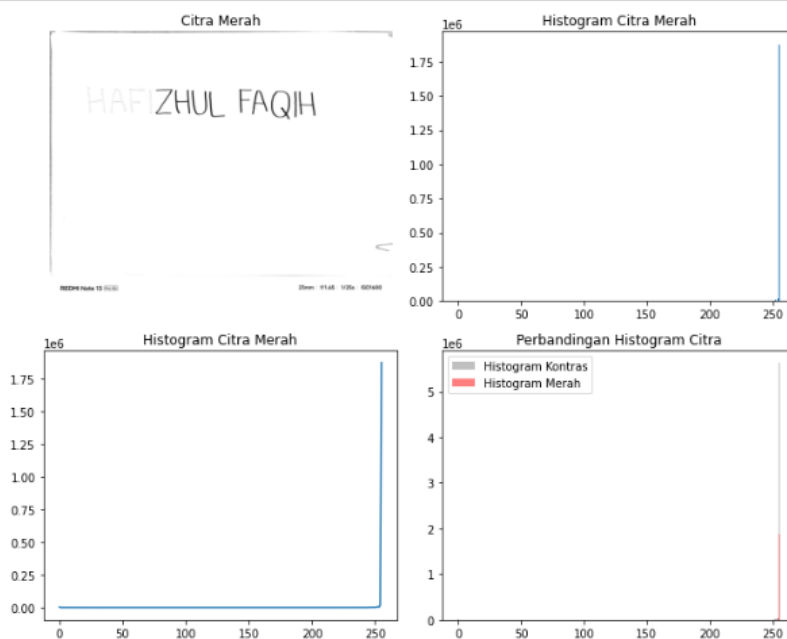
axs[0,0].imshow(r, cmap='gray')
axs[0,0].set_title("Citra Merah")
axs[0,0].axis('off')

axs[0,1].set_title("Histogram Citra Merah")
axs[0,1].hist(r.ravel(), 256, [0,256])

axs[1,0].set_title("Histogram Citra Merah")
hist_red = cv.calcHist([r],[0],None,[256],[0,256])
axs[1,0].plot(hist_red)

axs[1,1].hist(kontras.ravel(), bins=256, range=[0,256], color='gray', alpha=0.5, label="Histogram Kontras")
axs[1,1].hist(r.ravel(), bins=256, range=[0,256], color='red', alpha=0.5, label="Histogram Merah")
axs[1,1].set_title("Perbandingan Histogram Citra")
axs[1,1].legend()

plt.tight_layout()
plt.show()
```



Pertama adalah pemilihan kanal warna, karena sebuah gambar dibaca dalam *array* karena yang membedakan warna pada sebuah citra adalah dimensi ketiga, maka pada *variable* *r* akan merepresentasikan satu warna yaitu warna merah (2).

Kemudian warna biru yang ada pada dimensi *array* *r* akan dibuat menjadi *grayscale* dengan `cmap='gray'`

Kemudian pada tiap histogram yang ditampilkan sebenarnya membawa informasi yang sama, yaitu distribusi nilai *pixel* pada kanal warna merah. Namun dengan visualiasi yang berbeda.

Histogram pertama merupakan representasi distribusi nilai kanal warna merah dalam bentuk diagram balok, histogram kedua merupakan representasi distribusi nilai kanal merah dalam

diagram garis, dan histogram ketiga adalah perbandingan histogram merah yang dibuat *grayscale* dengan histogram yang merepresentasikan nilai warna dengan kontras (asli)

Pembuatan Citra Hijau

```
In [6]: g = gambar[:, :, 1]

fig, axs = plt.subplots(2,2, figsize=(10, 8))

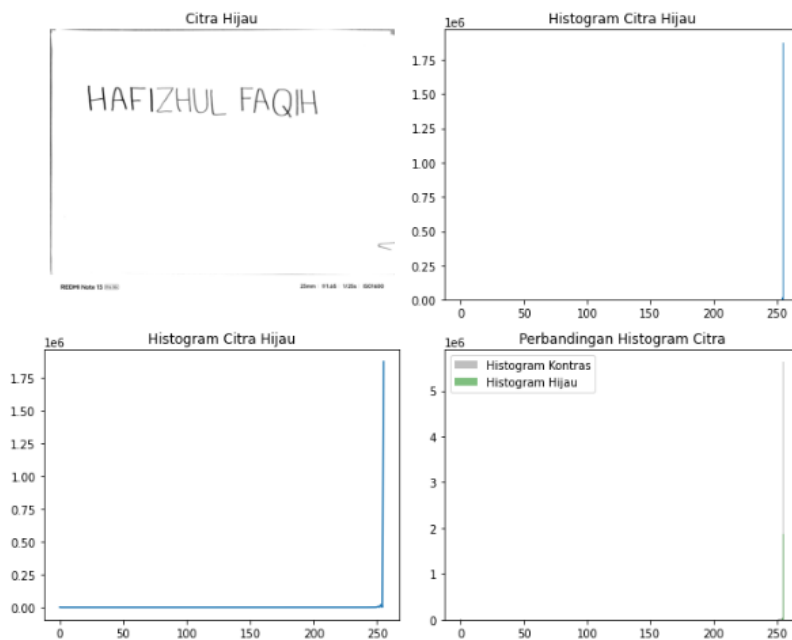
axs[0,0].imshow(g, cmap='gray')
axs[0,0].set_title("Citra Hijau")
axs[0,0].axis('off')

axs[0,1].set_title("Histogram Citra Hijau")
axs[0,1].hist(g.ravel(),256,[0,256])

axs[1,0].set_title("Histogram Citra Hijau")
hist_green = cv.calcHist([b],[0],None,[256],[0,256])
axs[1,0].plot(hist_green)

axs[1,1].hist(kontras.ravel(), bins=256, range=[0,256], color='gray', alpha=0.5, label="Histogram Kontras")
axs[1,1].hist(g.ravel(), bins=256, range=[0,256], color='green', alpha=0.5, label="Histogram Hijau")
axs[1,1].set_title("Perbandingan Histogram Citra")
axs[1,1].legend()

plt.tight_layout()
plt.show()
```



Pertama adalah pemilihan kanal warna, karena sebuah gambar dibaca dalam *array* karena yang membedakan warna pada sebuah citra adalah dimensi ketiga, maka pada *variable* *g* akan merepresentasikan satu warna yaitu warna hijau (1).

Kemudian warna hijau yang ada pada dimensi *array* *r* akan dibuat menjadi *grayscale* dengan *cmap='gray'*

Kemudian pada tiap histogram yang ditampilkan sebenarnya membawa informasi yang sama, yaitu distribusi nilai *pixel* pada kanal warna hijau. Namun dengan visualiasi yang berbeda.

Histogram pertama merupakan representasi distribusi nilai kanal warna hijau dalam bentuk diagram balok, histogram kedua merupakan representasi distribusi nilai kanal hijau dalam

diagram garis, dan histogram ketiga adalah perbandingan histogram hijau yang dibuat *grayscale* dengan histogram yang merepresentasikan nilai warna dengan kontras (asli)

Perbandingan Masing-Masing Citra Biru, Merah, dan Hijau

```
In [7]: fig, axs = plt.subplots(2,2, figsize=(10, 8))

axs[0,0].imshow(kontras, cmap='gray')
axs[0,0].set_title("Gambar Asli")
axs[0,0].axis('off')

axs[0,1].imshow(b, cmap='gray')
axs[0,1].set_title("Gambar Biru")
axs[0,1].axis('off')

axs[1,0].imshow(g, cmap='gray')
axs[1,0].set_title("Gambar Hijau")
axs[1,0].axis('off')

axs[1,1].imshow(r, cmap='gray')
axs[1,1].set_title("Gambar Merah")
axs[1,1].axis('off')

plt.tight_layout()
plt.show()
```



Kemudian ada perbandingan hasil dari pengolahan citra pada tiap kanal warna yang sebelumnya telah dibuat dengan model tata letak *tight*.

3.3.Pembahasan Soal Kedua

```
In [8]: gambar
Out[8]: array([[ 76, 131, 162],
               [ 80, 134, 165],
               [ 84, 135, 168],
               ...,
               [ 88, 164, 216],
               [ 94, 170, 222],
               [ 96, 175, 226]],

              [[ 74, 129, 160],
               [ 77, 131, 162],
               [ 82, 131, 163],
               ...,
               [ 88, 164, 216],
               [ 94, 170, 222],
               [ 94, 173, 224]],

              [[ 74, 128, 158],
               [ 74, 128, 158],
               [ 79, 129, 159],
               ...,
               [ 90, 166, 218],
               [ 93, 169, 221],
               [ 93, 172, 223]],

              ...,

              [[255, 255, 255],
               [255, 255, 255],
               [255, 255, 255],
               ...,
               [255, 255, 255],
               [255, 255, 255],
               [255, 255, 255]],

              [[255, 255, 255],
               [255, 255, 255],
               [255, 255, 255],
               ...,
               [255, 255, 255],
               [255, 255, 255],
               [255, 255, 255]],

              [[255, 255, 255],
               [255, 255, 255],
               [255, 255, 255],
               ...,
               [255, 255, 255],
               [255, 255, 255],
               [255, 255, 255]]], dtype=uint8)
```

Python membaca citra digital dalam bentuk *array*, sehingga saat mengetikkan *variable* gambar, yang muncul adalah *array* mentah yang dimensinya mewakili tinggi, lebar, dan warna dari gambar yang ada. Dan tipe datanya adalah `uint=8` yang memuat nilai *pixel* 0-255.

```

In [10]: b = gambar[:, :, 0]
         g = gambar[:, :, 1]
         r = gambar[:, :, 2]

In [11]: panjang = len(gambar)
         lebar = len(gambar[0])

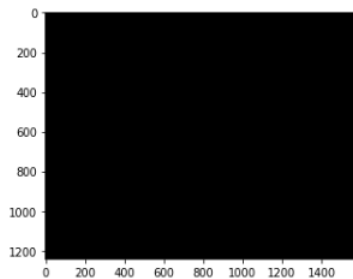
         citra_gray = np.zeros((panjang, lebar))

         for i in range(panjang):
             for j in range(lebar):
                 citra_gray[i,j] = 0.3989 * r[i,j] + 0.587 * g[i,j] + 0.1141 * b[i,j]
         citra_gray = citra_gray.astype(np.uint8)

In [12]: panjang = len(gambar)
         lebar = len(gambar[0])
         ambang = 255
         gambar_citra = np.zeros((panjang, lebar))
         for i in range(panjang):
             for j in range(lebar):
                 if citra_gray[i,j] > ambang:
                     gambar_citra[i,j] = 1
                 else:
                     gambar_citra[i,j] = 0
         plt.imshow(gambar_citra, cmap='gray')

Out[12]: <matplotlib.image.AxesImage at 0x2a4758e7190>

```



Masih sama seperti pembahasan soal pertama, pertama-tama masing-masing kanal warna akan dipisahkan menjadi tiga *variable* yang berisi warna biru, hijau, dan merah yang dibuat dalam bentuk inisial agar mudah dalam pemanggilan *variable* saat melakukan pengecekan warna pada citra.

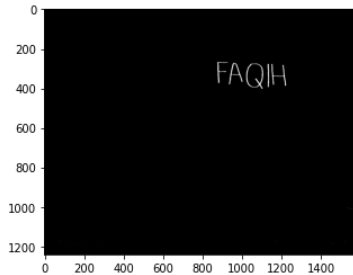
Kemudian mengkonversi citra berwarna ke *grayscale* secara manual menggunakan loop *pixel* per *pixel* dengan rumus pembobotan *luminance*.

Menerapkan *binerisasi* pada citra *grayscale*. Karena nilai ambang = 255 (nilai maksimum), tidak ada *pixel* yang melebihinya, sehingga seluruh citra hasil menjadi hitam (bernilai 0).

Nilai Ambang Biru

```
In [13]: ambang_bawah_biru = 95
ambang_atas_biru = 255
gambar_b = np.zeros((panjang, lebar), dtype=np.uint8)
for i in range(panjang):
    for j in range(lebar):
        if ambang_bawah_biru <= b[i, j] <= ambang_atas_biru and g[i, j] < 100 and r[i, j] < 100:
            gambar_b[i, j] = 1
        else:
            gambar_b[i, j] = 0
plt.imshow(gambar_b, cmap='gray')
```

Out[13]: <matplotlib.image.AxesImage at 0x2a4746b1790>



Untuk membuat kata yang berwarna biru muncul, pertama-tama perlu ada pembuatan batas ambang atas dan bawah pada kanal warna. Batas bawah nya adalah 95 dan batas atas nya adalah 255.

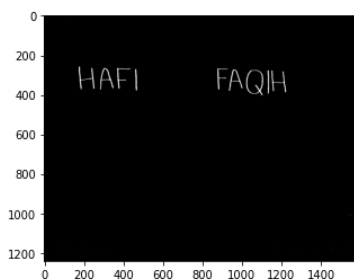
Pembuatan *variable* gambar_b untuk menampung *array* citra beserta tipe datanya.

Kemudian agar warna lain tidak muncul, G dan R harus < 100 . Kemudian *pixel* yang memenuhi syarat akan akan berwarna putih (1), sedangkan lainnya akan berwarna hitam (0). Pengecekan kondisi menggunakan *if statement* untuk memeriksa kondisi dan *looping* untuk melanjutkan pencarian kondisi pada tiap *pixel* nya sehingga semua *pixel* telah diperiksa.

Nilai Ambang Biru dan Merah

```
In [14]: ambang_bawah_merah = 120
ambang_atas_merah = 255
gambar_m = np.zeros((panjang, lebar), dtype=np.uint8)
for i in range(panjang):
    for j in range(lebar):
        biru = ambang_bawah_biru <= b[i, j] <= ambang_atas_biru and r[i, j] < 100 and g[i, j] < 100
        merah = ambang_bawah_merah <= r[i, j] <= ambang_atas_merah and g[i, j] < 100 and b[i, j] < 100
        if merah or biru:
            gambar_m[i, j] = 1
        else:
            gambar_m[i, j] = 0
plt.imshow(gambar_m, cmap='gray')
```

Out[14]: <matplotlib.image.AxesImage at 0x2a4731b9a90>



Untuk membuat warna merah dan biru muncul, langkah yang dilakukan kirang lebih sama dengan sebelumnya. Namun di sini karena kanal warna merah masih belum memiliki batas

ambang atas dan bawah, maka perlu dibuat *variable* yang menetapkan nilai batas ambang atas dan bawah pada kanal warna merah (bawah = 150, atas = 255).

Pembuatan *variable* gambar_m untuk menampung *array* citra beserta tipe datanya.

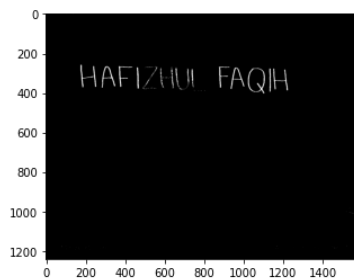
Untuk menampilkan warna merah dan biru juga menggunakan *if statement* dan *for looping* untuk menggabungkan dua kondisi *thresholding*, *pixel* dianggap positif jika biru atau merah. Hasilnya lebih banyak area terdeteksi dibanding hanya kanal biru saja.

Nilai Ambang Semua Warna

```
In [15]: ambang_bawah_hijau = 95
ambang_atas_hijau = 255
gambar_bgr = np.zeros((panjang, lebar), dtype=np.uint8)
for i in range(panjang):
    for j in range(lebar):
        biru = ambang_bawah_biru <= b[i, j] <= ambang_atas_biru and r[i, j] < 100 and g[i, j] < 100
        hijau = ambang_bawah_hijau <= g[i, j] <= ambang_atas_hijau and r[i, j] < 100 and b[i, j] < 100
        merah = ambang_bawah_merah <= r[i, j] <= ambang_atas_merah and g[i, j] < 100 and b[i, j] < 100
        if merah or biru or hijau:
            gambar_bgr[i, j] = 1
        else:
            gambar_bgr[i, j] = 0

plt.imshow(gambar_bgr, cmap='gray')
```

Out[15]: <matplotlib.image.AxesImage at 0x2a4731853d0>



Untuk menampilkan semua warna, maka perlu penambahan satu *variable* yang menampung batas atas dan bawah untuk warna hijau, karena yang telah ditetapkan batas atas dan bawah yang dibuat itu hanya merah dan biru. Sedangkan hijau belum.

Untuk pemanggilan *variable* yang berisi gabungan tiga warna, perlu pembuatan *variable* gambar_bgr.

Untuk memeriksa kondisi tiap *pixel* dalam citra, kembali menggunakan *if statement* dan *for looping* untuk Memperluas deteksi dengan menambahkan kondisi hijau. *Pixel* terdeteksi jika biru, hijau, atau merah. Menghasilkan area terdeteksi paling luas dari semua variasi *thresholding*.

Perbandingan Nilai Ambang Warna

```
In [16]: fig, axes = plt.subplots(2,2, figsize=(10, 8))

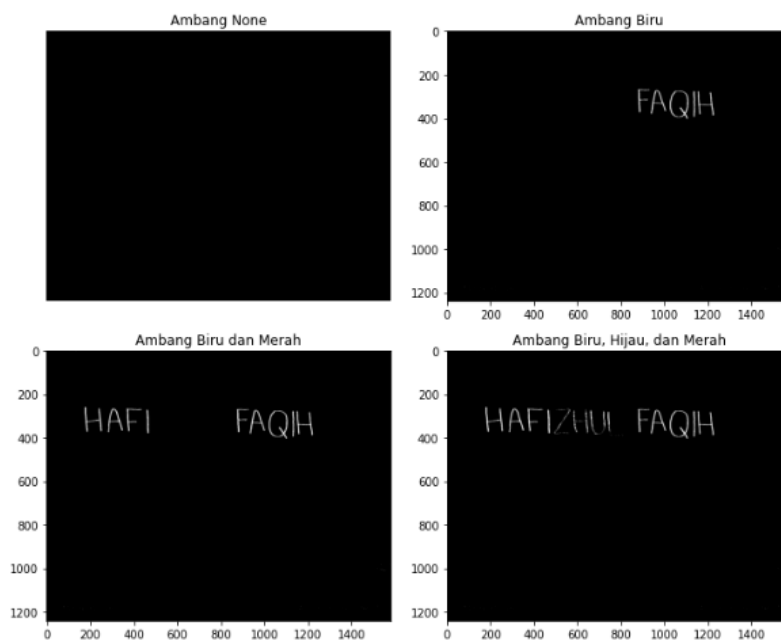
axes[0,0].imshow(gambar_citra, cmap='gray')
axes[0,0].set_title("Ambang None")
axes[0,0].axis('off')

axes[0,1].set_title("Ambang Biru")
axes[0,1].imshow(gambar_b, cmap='gray')

axes[1,0].set_title("Ambang Biru dan Merah")
axes[1,0].imshow(gambar_m, cmap='gray')

axes[1,1].set_title("Ambang Biru, Hijau, dan Merah")
axes[1,1].imshow(gambar_bgr, cmap='gray')

plt.tight_layout()
plt.show()
```



Perbandingan tiap hasil dari pengolahan citra dengan menggunakan batas atas dan bawah yang diberi judul untuk tiap hasilnya sesuai dengan hasil yang ditampilkan dengan tata letak *tight*.

3.4.Pembahasan Soal Ketiga

```
In [17]: img = cv.imread("FOTO/FOTO_BACKLIGHT.jpeg")
img
```

```
Out[17]: array([[[ 74,  74,  74],
 [ 72,  72,  72],
 [ 68,  68,  68],
 ...,
 [185, 173, 173],
 [185, 173, 173],
 [185, 173, 173]],
 [[ 68,  68,  68],
 [ 68,  68,  68],
 [ 66,  66,  66],
 ...,
 [185, 173, 173],
 [185, 173, 173],
 [185, 173, 173]],
 [[ 62,  62,  62],
 [ 63,  63,  63],
 [ 63,  63,  63],
 ...,
 [185, 173, 173],
 [185, 173, 173],
 [185, 173, 173]],
 ...,
 [[ 11,  13,  14],
 [ 11,  13,  14],
 [ 11,  13,  14],
 ...,
 [ 13,  13,  13],
 [ 13,  13,  13],
 [ 13,  13,  13]],
 [[ 11,  13,  14],
 [ 11,  13,  14],
 [ 11,  13,  14],
 ...,
 [ 13,  13,  13],
 [ 13,  13,  13],
 [ 13,  13,  13]],
 [[ 11,  13,  14],
 [ 11,  13,  14],
 [ 11,  13,  14],
 ...,
 [ 13,  13,  13],
 [ 13,  13,  13],
 [ 13,  13,  13]]], dtype=uint8)
```

Pembacaan nilai *array* dari gambar dengan *variable* *img*, yang berisi tinggi, lebar, dan kanal warna pada citra atau gambar foto *backlight*, dengan tipe data *unit8* yang berisi nilai *pixel* 0-255.

Foto Asli

```
In [20]: img_asli = cv.cvtColor(img, cv.COLOR_BGR2RGB)
plt.imshow(img_asli)
plt.axis("off")
```

```
Out[20]: (-0.5, 647.5, 1150.5, -0.5)
```



Pertama-tama perlu Membaca foto kedua (foto *backlight*), mengambil informasi dimensinya, lalu menyimpan jumlah baris dan kolom untuk digunakan dalam iterasi *loop* berikutnya.

Foto Abu

```
In [21]: img_abu = cv.cvtColor(img_asli, cv.COLOR_BGR2GRAY)
plt.imshow(img_abu, cmap='gray')
plt.axis('off')
```

```
Out[21]: (-0.5, 647.5, 1150.5, -0.5)
```



Untuk mengubah citra yang sebelumnya masih menggunakan kombinasi dari tiga warna (BGR) menjadi *grayscale* menggunakan fungsi yang tersedia dari *library* openCV, yaitu `cvtColor cv.COLOR_BGR2GRAY`.

Mencerahkan Foto

```
In [22]: beta = 17
img_cerah = np.zeros((baris, kolom, 3))

for x in range(baris):
    for y in range(kolom):
        gyx = img_abu[x,y] + beta
        img_cerah[x,y] = gyx

img_cerah = img_cerah.astype(np.uint8)

fig, axs = plt.subplots(2, figsize=(20,10))
axs[0].imshow(img_abu, cmap='gray')
axs[0].axis('off')
axs[1].imshow(img_cerah)
axs[1].axis('off')
plt.show()
```



Proses untuk meningkatkan kecerahan pada citra digital adalah dengan menjumlahkannya dengan nilai beta (17).

Menambahkan nilai $\beta = 17$ ke setiap *pixel* secara manual melalui loop. Hasilnya adalah citra yang lebih cerah dan semua *pixel* naik 17 intensitas.

Kontras Foto

```
In [23]: alpha = 1.05
img_kontras = np.zeros((baris, kolom, 3))

for x in range(baris):
    for y in range(kolom):
        gmx = img_abu[x,y] * alpha
        img_kontras[x,y] = gmx

img_kontras = img_kontras.astype(np.uint8)

fig, axs = plt.subplots(2, figsize=(20,10))
axs[0].imshow(img_abu, cmap='gray')
axs[0].axis('off')
axs[1].imshow(img_kontras)
axs[1].axis('off')
plt.show()
```



Berbeda dengan proses meningkatkan intensitas kecerahan pada citra digital yang sebelumnya menggunakan operasi penjumlahan, proses meningkatkan ketajaman atau kontras gambar menggunakan operasi perkalian.

Setiap *pixel* yang ada akan diperiksa menggunakan *for loop* dan mengalikannya dengan nilai alpha (1.05). sehingga hasilnya menjadi lebih konntras dibandingkan sebelumnya.

Mencerahkan dan Mengkontraskan Foto

In [24]: `print("202431150")`

202431150

```
In [25]: import numpy as np
import matplotlib.pyplot as plt

alpha = 1
beta = 15

img_hasil = (alpha * img_abu) + beta
img_hasil = np.clip(img_hasil, 0, 255)
img_hasil = img_hasil.astype(np.uint8)

fig, axs = plt.subplots(1, 2, figsize=(20,10))
axs[0].imshow(img_abu, cmap='gray')
axs[0].set_title("Citra Asli")
axs[0].axis('off')

axs[1].imshow(img_hasil, cmap='gray')
axs[1].set_title("Citra Hasil Kontras/Brightness")
axs[1].axis('off')

plt.show()
```



Untuk menggabungkan proses peningkatan kecerahan dan kontras, perlu Menerapkan rumus transformasi linear $g = \alpha \times f + \beta$. Yang artinya hasil citra baru adalah hasil perkalian nilai *pixel* dengan nilai alpha, kemudian dijumlahkan dengan nilai beta.

Yang di mana telah ditetapkan :

alpha = 1

beta = 15

karena bisa jadi hasil gambar barunya akan melewati batas (0-255), perlu fungsi `np.clip()` untuk memastikan nilai *pixel* tidak melebihi 255 atau di bawah 0.

Kemudian proses pada transformasi nilai *pixel* dengan rumus transformasi linear $g = \alpha \times f + \beta$ diterapkan pada setiap *pixel*, *for loop* digunakan agar tiap *pixel* pada citra bertransformasi menggunakan rumus transformasi linear.

Perbandingan Citra

```
In [26]: fig, axs = plt.subplots(2,2, figsize=(10,10))

axs[0,0].set_title("Foto Abu")
axs[0,0].imshow(img_abu, cmap='gray')
axs[0,0].axis('off')

axs[0,1].set_title("Foto Cerah")
axs[0,1].imshow(img_cerah, cmap='gray')
axs[0,1].axis('off')

axs[1,0].set_title("Foto Kontras")
axs[1,0].imshow(img_kontras, cmap='gray')
axs[1,0].axis('off')

axs[1,1].set_title("Foto Gabungan")
axs[1,1].imshow(img_hasil, cmap='gray')
axs[1,1].axis('off')

plt.tight_layout()
plt.show()
```



In []:

Menampilkan empat hasil akhir sebagai perbandingan antara citra asli dan berbagai hasil transformasi *enhancement* yang telah dilakukan.

Output perbandingan ditampilkan dalam subplot 2×2 yang memuat citra abu asli, citra setelah penambahan kecerahan, citra setelah peningkatan kontras, dan citra hasil transformasi gabungan *brightness* dan kontras.

BAB IV

PENUTUP

Berdasarkan landasan teori dan hasil praktikum UTS Pengolahan Citra Digital yang telah dilaksanakan, dapat ditarik kesimpulan sebagai berikut:

Pertama, citra digital merupakan representasi matriks tiga dimensi (tinggi $\times$ lebar $\times$ kanal warna) yang dapat dimanipulasi menggunakan *library* seperti OpenCV dan NumPy dalam Python. Pembacaan citra dengan OpenCV menghasilkan format BGR, sehingga diperlukan konversi ke RGB untuk visualisasi yang akurat menggunakan Matplotlib.

Kedua, pemisahan kanal warna (Biru, Hijau, Merah) memungkinkan analisis pada intensitas *pixel* pada masing-masing komponen warna. Histogram citra terbukti menjadi alat yang sangat berguna dalam menganalisis karakteristik visual citra secara kuantitatif, termasuk dalam memahami distribusi kecerahan dan dominasi warna dalam sebuah gambar.

Ketiga, konversi *grayscale* secara manual menggunakan rumus pembobotan *luminance* memberikan pemahaman mendalam tentang dasar matematis pengolahan citra.

Keempat, teknik segmentasi berbasis nilai ambang (*thresholding*) warna terbukti efektif untuk mengisolasi *pixel* dengan rentang warna tertentu dalam citra. Penggabungan kondisi *thresholding* dari beberapa kanal sekaligus menghasilkan segmentasi yang lebih selektif dan akurat dibandingkan *thresholding* pada satu kanal saja.

Kelima, peningkatan kecerahan dan kontras melalui transformasi linear ($g = \alpha \times f + \beta$) terbukti mampu memperbaiki kualitas visual citra *backlight*. Fungsi *clipping* sangat penting untuk menjaga nilai piksel dalam rentang valid 0–255 agar tidak terjadi overflow yang menghasilkan keanehan visual pada citra hasil.

DAFTAR PUSTAKA

Poynton, C. (2023). *Digital video and HD: Algorithms and interfaces* (2nd ed.). Morgan Kaufmann Publishers.

Sinha, P., & Bhattacharjee, P. (2022). An improved histogram-based image enhancement technique for color images using adaptive thresholding. *Journal of King Saud University – Computer and Information Sciences*, 34(9), 7218–7228. <https://doi.org/10.1016/j.jksuci.2021.08.030>

Szeliski, R. (2022). *Computer vision: Algorithms and applications* (2nd ed.). Springer. <https://doi.org/10.1007/978-3-030-34372-9>

Zhang, W., Tang, P., & Zhao, L. (2021). Remote sensing image scene classification using CNN-CapsNet. *Remote Sensing*, 11(5), 494. <https://doi.org/10.3390/rs11050494>